NAME: V Vishwa Teja ID NO: 2520090033

SKILL WEEK 8

1. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"command > file 2>&1\"\n", argv[0]);
        return 1;
    }

    char command[1024];
    strcpy(command, argv[1]);

    char *merge = strstr(command, "2>&1");

    if (merge == NULL)
    {
        printf("Combined redirection 2>&1 not found\n");
        return 1;
    }

    *merge = '\0';

    char *redirect = strchr(command, '>');

    if (redirect == NULL)
    {
        printf("Output redirection not found\n");
        return 1;
    }

    *redirect = '\0';

    char *output_file = redirect + 1;

    while (*output_file == ' ')
        output_file++;

    char *end = output_file + strlen(output_file) - 1;

    while (end > output_file && (*end == ' ' || *end == '\n'))
    {
        *end = '\0';
        end--;
```

```
}

int fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (fd < 0)
{
    perror("open");
    return 1;
}

char *cmd = command;

while (*cmd == ' ')
    cmd++;

char *args[64];
int count = 0;

char *token = strtok(cmd, " \t\n");

while (token != NULL && count < 63)
{
    args[count++] = token;
    token = strtok(NULL, " \t\n");
}

args[count] = NULL;

if (count == 0)
{
    close(fd);
    printf("No command specified\n");
    return 1;
}

pid_t pid = fork();

if (pid == 0)
{
    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("stdout");
        exit(1);
    }

    if (dup2(STDOUT_FILENO, STDERR_FILENO) < 0)
    {
        perror("stderr");
        exit(1);
```

```
        }

        close(fd);

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
        close(fd);
        return 1;
    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("Combined redirection completed.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill.8.1.c -o skill.8.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.8.1 "sh -c 'echo Standard output; echo Error output >&2' > combined.txt 2>&1"
Combined redirection completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat combined.txt
cat: combined.txt: No such file or directory
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.8.1 "ls missing_folder > combined.txt 2>&1"
Combined redirection completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat combined.txt
ls: cannot access 'missing_folder': No such file or directory
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

2. To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_ARGS 32

void parse_command(char *command, char *args[])
{
    int i = 0;
    char *token = strtok(command, " \t\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \t\n");
    }

    args[i] = NULL;
}

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"command1 | command2 > file\"\n", argv[0]);
        return 1;
    }

    char input[1024];
    strcpy(input, argv[1]);

    char *pipe_pos = strchr(input, '|');

    if (pipe_pos == NULL)
    {
        printf("Pipe not found\n");
        return 1;
    }

    *pipe_pos = '\0';

    char *left = input;
    char *right = pipe_pos + 1;

    char *redirect = strchr(right, '>');
```

```
    char *output_file = NULL;

    if (redirect != NULL)
    {
        *redirect = '\0';
        output_file = redirect + 1;

        while (*output_file == ' ')
            output_file++;

        char *end = output_file + strlen(output_file) - 1;

        while (end > output_file && (*end == ' ' || *end == '\n'))
        {
            *end = '\0';
            end--;
        }
    }

    while (*left == ' ')
        left++;

    while (*right == ' ')
        right++;

    char *left_args[MAX_ARGS];
    char *right_args[MAX_ARGS];

    parse_command(left, left_args);
    parse_command(right, right_args);

    if (left_args[0] == NULL || right_args[0] == NULL)
    {
        printf("Invalid pipeline syntax\n");
        return 1;
    }

    int pipefd[2];

    if (pipe(pipefd) < 0)
    {
        perror("pipe");
        return 1;
    }

    printf("Execution plan:\n");
    printf("1. Execute: %s\n", left_args[0]);
    printf("2. Send output through pipe\n");
```

```c
    printf("3. Execute: %s\n", right_args[0]);

    if (output_file != NULL)
        printf("4. Redirect final output to: %s\n", output_file);

    pid_t p1 = fork();

    if (p1 == 0)
    {
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execvp(left_args[0], left_args);

        perror("execvp");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 == 0)
    {
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        if (output_file != NULL)
        {
            int fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

            if (fd < 0)
            {
                perror("output");
                exit(1);
            }

            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        execvp(right_args[0], right_args);

        perror("execvp");
        exit(1);
    }
```

```c
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill.8.2.c -o skill.8.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.8.2 "ls | grep .c > result.txt"
Execution plan:
1. Execute: ls
2. Send output through pipe
3. Execute: grep
4. Redirect final output to: result.txt
Pipeline execution completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat result.txt
&2' > combined.txt
argument_passing.c
mmap_source.txt
practical10
practical10.2
practical10.2.c
practical11.1
practical11.1.c
practical11.2
practical11.2.c
practical12.1
practical12.1.c
practical12.2
practical12.2.c
practical12.2.prevention
practical12.2.prevention.c
practical2.1
practical2.1.c
practical2.1.txt
practical3.1
practical3.1.c
practical3.2
practical3.2.c
practical4.1
practical4.1.c
practical4.2
practical4.2.c
practical5.1
practical5.1.c
practical5.2
practical5.2.c
practical6.1
practical6.1.c
practical6.1_client
practical6.1_client.c
practical6.2
practical6.2.c
practical7.1
practical7.1.c
practical8.1
practical8.1.c
practical8.2
```

```
practical8.2.c
practical9.1
practical9.1.c
practical9.1.standard
practical9.1.standard.c
practical9.2
practical9.2.c
practical_fixed
practical_fixed.c
problem1.c
protected
skill.7.1.c
skill.7.2.c
skill.7.3.c
skill.7.4.c
skill.8.1.c
skill.8.2.c
skill1.1.c
skill1.2.c
skill1.3.c
skill2.1.c
skill2.2.c
skill2.3.c
skill2.4.c
skill2.c
skill3.1.c
skill3.2.c
skill3.3.c
skill3.4.c
skill3.c
skill3main2.c
skill4.1.c
skill4.2.c
skill4.3.c
skill4.4.c
skill5.1.c
skill5.2.c
skill5.3.c
skill5.4.c
skill6.1.c
skill6.2.c
source.txt
sum1.c
task2.c
vishwa.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.8.2 "ps | grep bash"
Execution plan:
1. Execute: ps
2. Send output through pipe
3. Execute: grep
    341 pts/0    00:00:00 bash
Pipeline execution completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

3. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"command &\"\n", argv[0]);
        return 1;
    }

    char command[1024];
    strcpy(command, argv[1]);

    char *end = command + strlen(command) - 1;

    while (end > command && (*end == ' ' || *end == '\n'))
    {
        *end = '\0';
        end--;
    }

    if (*end != '&')
    {
        printf("Background operator & not found\n");
        return 1;
    }

    *end = '\0';

    char *cmd = command;

    while (*cmd == ' ')
        cmd++;

    char *args[64];
    int count = 0;

    char *token = strtok(cmd, " \t\n");

    while (token != NULL && count < 63)
    {
        args[count++] = token;
        token = strtok(NULL, " \t\n");
    }
```

```c
    args[count] = NULL;

    if (count == 0)
    {
        printf("No command specified\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid == 0)
    {
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    printf("[1] Background process started with PID %d\n", pid);
    printf("Parent continues immediately.\n");

    sleep(1);

    int status;

    pid_t result = waitpid(pid, &status, WNOHANG);

    if (result == 0)
    {
        printf("[1] Job %d is still running\n", pid);
    }
    else
    {
        printf("[1] Job %d completed\n", pid);
    }

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill.8.3.c -o skill.8.3
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.8.3 "sleep 5 &"
[1] Background process started with PID 1306
Parent continues immediately.
[1] Job 1306 is still running
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.8.3 "sleep 1 &"
[1] Background process started with PID 1317
Parent continues immediately.
[1] Job 1317 is still running
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

4. To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct
{
    int id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, pid_t pgid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full\n");
        return;
    }

    jobs[job_count].id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;

    strncpy(jobs[job_count].command, command, 99);
    jobs[job_count].command[99] = '\0';

    strcpy(jobs[job_count].state, "Running");

    printf("[%d] %d Running %s\n",
           jobs[job_count].id,
           jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
```

```
        int status;

        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result > 0)
        {
            strcpy(jobs[i].state, "Done");
        }
    }
}

void remove_completed_jobs()
{
    for (int i = 0; i < job_count;)
    {
        if (strcmp(jobs[i].state, "Done") == 0)
        {
            for (int j = i; j < job_count - 1; j++)
            {
                jobs[j] = jobs[j + 1];
            }

            job_count--;
        }
        else
        {
            i++;
        }
    }
}

void display_jobs()
{
    update_jobs();

    printf("\nJob Table\n");
    printf("ID\tPID\tPGID\tState\tCommand\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%d\t%d\t%d\t%s\t%s\n",
               jobs[i].id,
               jobs[i].pid,
               jobs[i].pgid,
               jobs[i].state,
               jobs[i].command);
    }

    printf("\n");
```

```c
}

int main()
{
    char command[100];

    printf("Job Manager\n");
    printf("Commands: sleep 5, sleep 10, jobs, clean, exit\n");

    while (1)
    {
        printf("job-shell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "jobs") == 0)
        {
            display_jobs();
            continue;
        }

        if (strcmp(command, "clean") == 0)
        {
            update_jobs();
            remove_completed_jobs();
            display_jobs();
            continue;
        }

        char command_copy[100];
        strcpy(command_copy, command);

        char *args[20];
        int count = 0;

        char *token = strtok(command, " ");

        while (token != NULL && count < 19)
        {
            args[count++] = token;
            token = strtok(NULL, " ");
        }
```

```c
        args[count] = NULL;

        if (count == 0)
            continue;

        pid_t pid = fork();

        if (pid == 0)
        {
            setpgid(0, 0);
            execvp(args[0], args);
            perror("execvp");
            exit(1);
        }

        if (pid < 0)
        {
            perror("fork");
            continue;
        }

        setpgid(pid, pid);

        add_job(pid, pid, command_copy);
    }

    update_jobs();

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill.8.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill.8.4.c -o skill.8.4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill.8.4
Job Manager
Commands: sleep 5, sleep 10, jobs, clean, exit
job-shell> sleep 10
[1] 1457 Running sleep 10
job-shell> jobs

Job Table
ID      PID     PGID    State   Command
1       1457    1457    Running sleep 10

job-shell> jobs

Job Table
ID      PID     PGID    State   Command
1       1457    1457    Done    sleep 10

job-shell> clean

Job Table
ID      PID     PGID    State   Command

job-shell> exit
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```